PDF Filler Integration Guide

For Claude Code — Wiring pdf_filler.py into the Orientation Intake App

Step 1: Add the files

1. Place `pdf_filler.py` in `app/services/pdf_filler.py`
2. Create directory `templates/county_forms/`
3. Place these three blank PDFs in that directory (exact filenames required):
   - `Attachment_V_WIOA_Applicant_Acknowledgement_Statements.pdf`
   - `Attachment_IV_WIOA_Complaint_Resolutions__Participant_Acceptance_Form.pdf`
   - `code_of_conduct.pdf`

Step 2: Update `app/services/__init__.py`

Add the import:

```
from app.services.pdf_filler import fill_all_county_forms, fill_attachment_v,
fill_attachment_iv, fill_code_of_conduct
```

Step 3: Update `requirements.txt`

The module uses `pypdf` and `reportlab` — both should already be in your requirements. Verify they're listed:

```
pypdf
reportlab
Pillow
```

Step 4: Wire into zip_builder.py

In `build_submission_zip()`, after packaging the existing forms and documents, call `fill_all_county_forms()` and add the resulting PDFs to the zip.

The signature bytes are in `participant_data["signature"]["image_bytes"]`.

```
from app.services.pdf_filler import fill_all_county_forms

# Inside build_submission_zip(), after existing form packaging:
signature_bytes = participant_data.get("signature", {}).get("image_bytes")
if signature_bytes:
    county_forms = fill_all_county_forms(
        participant_data=participant_data,
        signature_bytes=signature_bytes,
        emergency_contact=participant_data.get("emergency_contact"),
        staff_name="",  # Can be populated from session state if collected
```

```
    )

    name_prefix = f"
{participant_data['last_name']}_{participant_data['first_name']}"

    for form_name, pdf_bytes in county_forms.items():
        filename = f"county_forms/{name_prefix}_{form_name}.pdf"
        zf.writestr(filename, pdf_bytes)
```

Step 5: Collect emergency contact data

Attachment V requires emergency contact info. You need a form step in the participant flow that collects:

- Contact name
- Street address
- City
- Zip code
- Phone number
- Whether the contact is a government employee (determines Box 1 vs Box 2)

Store this in `participant_data["emergency_contact"]` as a dict:

```
{
    "name": "Jane Doe",
    "street": "123 Main St",
    "city": "Los Angeles",
    "zip": "90001",
    "phone": "(626) 555-1234",
    "is_government": False,  # True = Box 1, False = Box 2
}
```

Updated Zip Structure

After integration, the zip will look like:

```
Garcia_Maria_abc123_20260223.zip
├── documents/
│   ├── Garcia_Maria_government_id.pdf
│   └── ...
├── forms/
│   ├── Garcia_Maria_form_01_program_agreement.pdf
│   └── ... (14 forms)
├── county_forms/                          ← NEW
│   ├── Garcia_Maria_attachment_v.pdf
│   ├── Garcia_Maria_attachment_iv.pdf
│   └── Garcia_Maria_code_of_conduct.pdf
├── signature/
```

```
│   └── Garcia_Maria_signature.png
└── Garcia_Maria_info.txt
```

How It Works (for reference)

- **Attachment V**: Has native fillable form fields (AcroForm). Uses `pypdf` to inject values directly into existing fields. Signature overlaid with ReportLab.
- **Attachment IV**: No fillable fields. Text overlaid at precise coordinates using ReportLab canvas, then merged onto original PDF. Signature overlaid separately.
- **Code of Conduct**: Scanned flat PDF. Same overlay approach as Attachment IV — date text and signature placed at coordinates.

All operations are in-memory. No temp files persist. Compatible with zero-persistence architecture.